

编译器视角：理解他，并应用他

```
1 #stm32f103xb.h
2 #define __IO volatile
3 #define __IO volatile
4
5 typedef struct
6 {
7     __IO uint32_t CRL;
8     __IO uint32_t CRH;
9     __IO uint32_t IDR;
10    __IO uint32_t ODR;
11    __IO uint32_t BSRR;
12    __IO uint32_t BRR;
13    __IO uint32_t LCKR;
14 } GPIO_TypeDef;
```

```
/*< Defines 'write only' permissions */
/*< Defines 'read / write' permissions */
```

```
1 #cmsis_gcc.h
2 #define __NO_RETURN
3 #define __USED
4 #define __WEAK
5 #define __PACKED
6 #define __PACKED_STRUCT
7 #define __PACKED_UNION

__attribute__((__noreturn__))
__attribute__((__used__))
__attribute__((__weak__))
__attribute__((__packed, __aligned(1)))
struct __attribute__((__packed, __aligned(1)))
union __attribute__((__packed, __aligned(1)))
```

```
1 #include/linux/init.h
2 #ifndef MODULE
3 // 省略
4 #define module_init(x) __initcall(x);
5 // 省略
6 #else
7
8 #define module_init(x) __initcall(x);
9 |
10 --> #define __initcall(fn) device_initcall(fn)
11 |
12 --> #define device_initcall(fn) __define_initcall(fn, 6)
13 |
14 --> #define __define_initcall(fn, id) \
15     static initcall_t __initcall_##fn##id __used \
16     __attribute__((__section__(".initcall" #id ".init"))) = fn
```